Project 3.1 — PolicyPal-HR-Policy-Assistant-With-Citations-And-Approval

A practice twin of Project 3 — DocuMind. Same skills — a RAG agent on a LangGraph graph that **retrieves** before it answers, **grounds** every answer in the retrieved text with a citation, says **"not in the policies"** instead of guessing, and **pauses for a human to approve** any consequential action before it happens (saved state, resumed in a separate run). A **new, real-world job**. **There are no steps here on purpose.** Read the story, build it yourself, and only open the hints or the solution if you get stuck.

Type: RAG agent (3 roles) on a LangGraph graph with human-in-the-loop · **Stack:** Python, LangGraph (+ SQLite checkpointer), LangChain, Chroma (embeddings/retrieval), Pydantic · **Level:** Intermediate

> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

An employee messages HR: *"How many days of paid leave do I get, and can you file one for next Monday?"* A careless assistant answers from memory — and now someone's planning around a made-up number. A good one does two things differently: it **looks the answer up in the actual HR policies** and quotes where it came from, and before it *files* anything on someone's behalf, it **stops and asks a human to approve**.

You are going to build **PolicyPal**: an HR assistant that is honest about what it knows and careful about what it does.

- It **retrieves** the most relevant policy passages for the question before answering — never from the model's memory.
- It **answers grounded**, with a citation to the policy file the answer came from. If the policies don't cover the question, it says **"that isn't in the HR policies"** instead of inventing a number.
- When the request is an **action** ("file a leave request"), it doesn't just do it. It pauses, shows a human a checklist of exactly what it's about to do, and waits for **approval** — and because the paused state is *saved*, a completely separate run can approve and finish it.

The one lesson of this project: an assistant people trust with real policies and real actions must (1) ground every answer in retrieved evidence it can cite, and (2) never take a consequential action without a human in the loop.

How it works, from the question to the answer (picture the whole machine)

Read this once and the whole project should form in your head.

Everything runs on a small **LangGraph** graph whose state carries the question, the retrieved passages, the answer, and — for actions — an approval decision. When you ask something, the **retriever** role first decides whether this is a *question* or an *action request*, then searches the policy collection (built once by embedding the HR documents into Chroma) and keeps only passages that are actually close enough to be relevant — measured by the embedding **distance** (lower = closer). If nothing is close enough, it marks the run "not found."

For a plain question, the **reasoner** role writes the answer **from the retrieved passages only**, and adds a citation to the file each passage came from. If the retriever found nothing relevant, the reasoner doesn't try — it returns the honest "not in the policies" answer. You see the answer *and* where it came from.

For an **action** ("file a leave request for Monday"), the graph routes to the **approval** role. It builds a short checklist — *what* it will do, for *whom*, *when* — and then calls LangGraph's `interrupt()`, which **pauses the whole graph and saves its state**. Nothing happens yet. A human (you) reviews the checklist and resumes the run with *yes* or *no*. Because the state was saved by the checkpointer, the approval can come from a **different process entirely** — you start the run in one terminal, stop at the pause, and finish it from another. On *yes*, the action is recorded; on *no*, it's cancelled and the run says so.

That's the whole machine: **retrieve → (is it relevant? no → "not found") → answer grounded with a citation, OR → build a checklist → pause and save → a human approves or rejects → act only then.**

What you are building (the pieces)

- A **RAG layer**: chunk the HR documents, embed them into a Chroma collection built once, and a `retrieve()` that returns passages with their **distance** score and source file.
- A **LangGraph graph** with a saved-state **checkpointer** (SQLite), so a run can pause and be resumed later — even from another process.
- A **retriever** role that routes question-vs-action, searches, and drops passages above a distance cutoff (the "not found" signal).
- A **reasoner** role that answers from retrieved passages only, with a citation — and refuses cleanly when nothing was retrieved.
- An **approval** role that builds a checklist and calls `interrupt()` before any action; a *yes* records it, a *no* cancels it.

Inputs: an HR question or an action request. **Outputs:** a grounded, cited answer; an honest "not found"; or an action taken only after human approval. **Rules:** answers cite their source; nothing is invented; no action happens without a human *yes*.

A real example to build and test against

- *"How many paid leave days do I get?"* → retrieves the leave policy → grounded answer with a citation to `leave_policy.md`.
- *"What's the Wi-Fi password in the Berlin office?"* → nothing relevant retrieved → "that isn't in the HR policies."
- *"File a leave request for next Monday."* → routes to approval → pauses with a checklist → resumed with *yes* (recorded) or *no* (cancelled).
- The same action started in one run and **approved in a separate run** — proving the state really was saved.

How to practice this

1. Read this story. **Don't open the solution yet.**
2. Plan it on paper: the graph state and nodes, the retrieval distance cutoff, and exactly where the `interrupt()` pause sits.
3. Build it yourself. A good order: the smallest graph that answers → saved state + a pause you can resume → retrieval + routing → grounded citing reasoner → the approval gate. Your call.
4. Test it on the real example: a grounded question, a not-found question, an approved action, and a rejected one — plus pause-in-one-run, resume-in-another.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

This is the shape of every assistant that sits on top of a company's own documents and can *do* things: an HR bot, an IT helpdesk, a customer-support agent over a knowledge base, an internal ops assistant. Two habits make it trustworthy, and they're the whole point: **ground every answer in retrieved evidence you can cite** (so it can't confidently invent a policy), and **put a human in the loop before any consequential action** (so it can't quietly do something on someone's behalf). The saved-state pause is how real systems hand an approval to a person who might be on a different machine, minutes or hours later. Build PolicyPal from this story and you can build any grounded, human-approved agent.

Done when

- A covered question gets a grounded answer with a citation; an uncovered one gets "not in the policies" (no invention).
- An action request pauses for approval; *yes* records it, *no* cancels it.
- A run paused in one process can be resumed and finished in another (the state is really saved).
- Your tests pass: settings, chunking, retrieval + the distance cutoff, routing, and the approval checklist (no API key), plus a live grounded/not-found/approval run.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.